arXiv:2102.11872v1 [cs.LG] 23 Feb 2021

CAC: A Clustering Based Framework for Classification

Shivin Srivastava[1] **Siddharth Bhatia**[1] **Lingxiao Huang**[2] **Lim Jun Heng**[3] **Kenji Kawaguchi**[4]
Vaibhav Rajan[1]

[1]National University of Singapore
[2]Huawei TCS Lab
[3] Nanyang Technological University
[4] Harvard University

Abstract

In data containing heterogeneous subpopulations, classification performance benefits from incorporating the knowledge of cluster structure in the classifier. Previous methods for such combined clustering and classification either are classifier-specific and not generic, or independently perform clustering and classifier training, which may not form clusters that can potentially benefit classifier performance. The question of how to perform clustering to improve the performance of classifiers trained on the clusters has received scant attention in previous literature despite its importance in several real-world applications. In this paper, we theoretically analyze when and how clustering may help in obtaining accurate classifiers. We design a simple, efficient, and generic framework called **C**lassification **A**ware **C**lustering (CAC), to find clusters that are well suited for being used as training datasets by classifiers for each underlying subpopulation. Our experiments on synthetic and real benchmark datasets demonstrate the efficacy of CAC over previous methods for combined clustering and classification.

1 INTRODUCTION

Many real datasets have complex underlying structures such as clusters and intrinsic manifolds. For classification tasks on such data, linear classifiers are unable to learn well because of the inherent non-linear data distribution. Non-linear classifiers, such as (deep) neural networks can perform well in such cases, but require large training data sets to achieve good generalization. When clusters are found or suspected in the data, it is well known that classification performance benefits from incorporating knowledge of such cluster structure in the model, for both linear and non-linear classifiers [Qian et al., 2012, Gu and Han, 2013, Chakraborty, 2017, Li et al., 2020].

As a specific example, consider the problem of predicting the risk of disease using patients' clinical data. Patient populations, even for a single disease, show significant clinical heterogeneity. As a result, subpopulations (called subtypes) having relatively homogeneous clinical characteristics can often be found in the data. Models that do not consider underlying subtypes may be consistently underestimating or overestimating the risks in specific subtypes [Snyderman, 2012], and risk models that account for subtypes have been found to be more effective [Alaa et al., 2016, Suresh et al., 2018, Li et al., 2020]. Further, the discovered subtypes provide clinical interpretability. For instance, in the case of the life-threatening complication sepsis, Ibrahim et al. [2020] discover four subtypes through clustering and train classifiers on each of them to build an effective risk model. The feature sets for each classifier show direct clinical relevance with respect to each subtype. Similar examples illustrate the benefit of clustering to build classifiers in other domains, e.g., in image processing [Kim and Cipolla, 2008, Peikari et al., 2018] and natural language processing [Alsmadi and Alhami, 2015].

Previous works on using clustering for classification broadly fall into three categories. The first group of techniques modify specific classification techniques to account for the underlying clusters, e.g., Clustered SVMs [Gu and Han, 2013]. These techniques are closely tied to the modified classifier and inherit both its modeling strengths and weaknesses. The second group of methods attempts to cluster and train one or more classifiers simultaneously using various approaches, e.g., ensemble-based methods [Chakraborty, 2017]. The third and most common approach is to first cluster the data and then independently train classifiers on each cluster. Such a 'cluster-then-predict' approach may not form clusters in a manner that benefits the performance of the classifiers trained on the clusters. This is also a limitation of many techniques in the second group where clustering and classifier training occur independently. To our knowledge, the

question of how to perform clustering to improve the performance of classifiers trained on the clusters has not been addressed by previous literature.

In this paper, we first theoretically investigate the fundamental question of when and how can clustering help in obtaining accurate classifiers. Our analysis yields novel insights into the benefits of using simple classifiers trained on clusters compared to simple or complex (with respect to Rademacher complexity) classifiers trained without clustering. It provides clues on when such a clustering-based approach may not improve the subsequent classification. Finally, it also motivates the design of our clustering-based framework for classification, called Classification Aware Clustering (CAC).

The CAC framework can be viewed as a clustering algorithm that by design finds clusters that are intended to be used as training datasets by classifiers. The key idea of our approach is to induce class separability within each cluster. This is affected by designing a cost function that can be used within the optimization framework of k-means. However, finding clusters with this cost function using the popular Lloyd's algorithm may not guarantee convergence. Therefore, we design an approach based on Hartigan's method [Wong and Hartigan, 1979] and prove that it converges. Our design is generic and not classifier-specific, and leads to an efficient iterative method to find the clusters. We analyze the sensitivity of CAC to various data characteristics such as class separation, intercluster distance, and the number of clusters using synthetic data. Finally, we empirically study the performance of CAC on benchmark datasets, including two large clinical datasets. We find that the use of CAC leads to classification performance that is superior or comparable to previous methods for combined clustering and classification.

In summary, our contributions are:

- Our theoretical analysis provides insights into the fundamental question of how clustering can aid in potentially improving the performance of classifiers trained on the clusters.
- We design a novel algorithmic framework, Classification Aware Clustering (CAC) to find clusters in data with the aim of improving the performance of classifiers trained on the clusters. We prove its convergence. CAC is simple, efficient, and can be used with any classifier.
- We analyze the sensitivity of CAC to various algorithm parameters and cluster characteristics in the data using synthetic data. Our findings lend support to our theoretical analysis.
- Our experimental results on benchmark datasets including large clinical datasets, show that CAC yields higher improvement in classification performance compared to previous approaches for combined clustering and classification.

Reproducibility: Our code and datasets are publicly available at https://github.com/shivin9/CAC.

2 RELATED WORK

The idea of training multiple local classifiers on portions of the dataset has been extensively studied for specific classifiers [Hess et al., 2020, Elouedi et al., 2014, Vincent and Hansen, 2014, Visentin et al., 2019, Mahfouz et al., 2020, Beluch et al., 2018, Titov et al., 2010, Hao et al., 2018]. For instance, Qi et al. [2011] present a novel locality-sensitive support vector machine (LSSVM) for the image retrieval problem. Locally Linear SVM [Torr, 2011] has a smooth decision boundary and bounded curvature. The authors show how functions defining the classifier can be approximated using any local coding scheme. Fu et al. [2010] propose Mixed SVMs which partition the feature space into subregions of linearly separable data points and learn a Linear SVM for each of these regions. Gu and Han [2013] propose Clustered Support Vector Machines (CSVMs) which tackles the problem in a divide-and-conquer manner, thus avoiding the need for computationally expensive kernel SVMs. CSVM groups the data into several clusters, after which it trains a linear SVM in each cluster to separate the data locally. Additionally, CSVM has a global regularization, which requires the weight vector of each locally linear SVM to align with a global weight vector. There are several works that use classifiers in the medical domain [Tanwani and Farooq, 2009, Hussain et al., 2017, Patel et al., 2009, Heider et al., 2013, Silva et al., 2014, Ambrosen et al., 2014, Jiang et al., 2019].

There are other related works, where the aim is to find suitable representations of raw data such that the performance in downstream prediction tasks is better when they use the learned embeddings. Large Margin Nearest Neighbour (LMNN) [Weinberger and Saul, 2009] introduced the much-celebrated triplet loss. In this work, the authors find a data-specific Mahalanobis distance embedding for the data, subject to the constraint that k similarly labeled points are mapped in the neighborhood of every point. The transformed data is more amenable for a k Nearest Neighbour classifier to work with. Song et al. [2017] extend LMNN to make it parameter-free. Recently, there has been a surge of interest in contrastive learning [Sasaki et al., 2020, Durkan et al., 2020] as well where the core idea is to group similar/dissimilar representations of data into similar/dissimilar groups. The purpose of doing this is to generate representations that are beneficial for supervised classifiers.

Some Bayesian approaches have been developed for combined classification and clustering, e.g., [Qian et al., 2012, Cai et al., 2009] but they are computationally expensive. To

our knowledge, the closest and most recent works in spirit and purpose to our work are the EC3 and DMNN algorithms. Chakraborty [2017] designed the EC3 algorithm which is based on a systematic combination of multiple classification and clustering methods combined via a convex optimization function.

EC3 uses 6 base classifiers: Decision Tree, Naive Bayes, K-nearest neighbor, logistic regression, SVM, and stochastic gradient descent classifier. They also consider 3 clustering methods DBSCAN, Hierarchical, and k-Means.

The Deep Mixture Neural Network (DMNN) was proposed by Li et al. [2020] for simultaneous patient subtyping and risk prediction. It consists of an embedding network with gating and a user-defined number of local predictive networks. The gating network computes high-level feature representations of raw input features, which are then clustered to find patient cohorts. Local predictor networks are then trained to predict the outcomes for their respective cohorts. The subgroup-specific sets of features learned by the local features provide valuable information from which we can reason about the cluster phenotype. Thus, DMNN represents the 'cluster-then-predict' type of approach as it uses local predictor networks to classify subpopulations in the data.

3 PROBLEM STATEMENT

Let $X = (x_1, x_2, \cdots, x_n)$ be a training dataset with n records $x_i \in \mathbb{R}^d$ together with binary labels $y_i \in \{0,1\}$. Given an integer $k \geq 1$, our goal is to:

1. Divide X into k non-empty, distinct clusters $C_1, C_2, \cdots, C_k$, and
2. Obtain a collection of k classifiers $f_1, f_2, \cdots, f_k$ where each $f_j : C_j \to \{0,1\}^{|C_j|}$ is a classifier trained on all $x_i \in C_j$.

There is no restriction on the choice of the classification algorithm, which is determined apriori by the user.

4 CAN CLUSTERING AID CLASSIFICATION?

We first theoretically analyze when and how can such clustering potentially help in obtaining accurate classifiers. Our analysis yields crucial insights into developing our new framework.

Fix $j \in \{1, \ldots, k\}$ and let $S = \{(x_i, y_i)\}_{i=1}^m$ be the training dataset from the j-th cluster with $x \in \mathscr{X}_j$ and $y \in \mathscr{Y}_j$ s.t. $\mathscr{X}_j = \{x : x \in C_j\}$ and $\mathscr{Y}_j = \{y : y \in C_j\}$ Then, the following theorem — a direct application of a previous result [Bartlett and Mendelson, 2002, Mohri et al., 2012] to our problem — suggests a potential benefit of clustering in terms of the expected error $\mathbb{E}_{x,y}[\ell(f(x), y)]$ for unseen data:

Theorem 1. *Let $\mathscr{F}_j$ be a set of maps $x \mapsto f(x)$, and the function $q \mapsto \ell(q, y)$ be a λ-uniformly bounded function for any $q \in \{f(x) : f \in \mathscr{F}_j, x \in \mathscr{X}_j\}$ and $y \in \mathscr{Y}_j$. Then, for any $\delta > 0$, with probability at least $1 - \delta$ over an i.i.d. draw of m i.i.d. samples $((x_i, y_i))_{i=1}^m$, the following holds: for all maps $f \in \mathscr{F}_j$,*

$$\mathbb{E}_{x,y}[\ell(f(x), y)] \leq \frac{1}{m}\sum_{i=1}^{m} \ell(f(x_i), y_i) + 2\mathscr{R}_m(\ell \circ \mathscr{F}_j) + \lambda\sqrt{\frac{\ln(1/\delta)}{2m}}, \tag{1}$$

where $\mathscr{R}_m(\ell \circ \mathscr{F}_j) := \mathbb{E}_{S,\sigma}[\sup_{f \in \mathscr{F}_j} \frac{1}{m}\sum_{i=1}^m \sigma_i \ell(f(x_i), y_i)]$ where $\sigma_1, \ldots, \sigma_m$ are independent uniform random variables taking values in $\{-1, 1\}$.

Proof. We present the proof in Appendix A.1. □

Theorem 1 shows that the expected error $\mathbb{E}_{x,y}[\ell(f(x), y)]$ for unseen data is bounded by three terms: the training error $\frac{1}{m}\sum_{i=1}^m \ell(f(x_i), y_i)$, the Rademacher complexity $\mathscr{R}_m(\ell \circ \mathscr{F}_j)$ of the set of classifiers $\mathscr{F}_j$ on the j-th cluster, and the last term $O(\frac{\ln(1/\delta)}{2m})$ in equation (1). To apply Theorem 1 to models without clustering, we can set the j-th cluster to contain all the data points with $m = n$.

The upper bound can be used to analyze the benefit of using a simple model (e.g., a linear model) *with* clustering compared to two cases of (i) a simple model and (ii) a complex model (e.g., a deep neural network), both *without* clustering. For a simple model *without* clustering, the training error term $\frac{1}{m}\sum_{i=1}^m \ell(f(x_i), y_i)$ in equation (1) can be large, because a simple model may not sufficiently separate training data points to minimize the training error. In the case of a complex model *without* clustering, the training error can be small but the Rademacher complexity $\mathscr{R}_m(\ell \circ \mathscr{F}_j)$ tends to be large. A simple model *with* clustering can potentially trade-off these by using a simple model to minimize the Rademacher complexity $\mathscr{R}_m(\ell \circ \mathscr{F}_j)$ while making the training data in the cluster to be separable to minimize the training error $\frac{1}{m}\sum_{i=1}^m \ell(f(x_i), y_i)$.

Theorem 1 also shows that such a clustering-based approach may also not help in some cases in improving classification. Using the clustering approach decreases the value of m and thus increases the last term $O\left(\frac{\ln(1/\delta)}{2m}\right)$ in Equation (1), when compared with models without clustering. Therefore, using a simple model with clustering can be potentially advantageous when the first and second terms in Equation (1) are dominant, but may not be beneficial if the last term is dominant. Here, the Rademacher complexity term $\mathscr{R}_m(\ell \circ \mathscr{F}_j)$ is also dependent on the sample size of the cluster m as it can be typically bounded by $O(\sqrt{\frac{M}{m}})$ for both linear models and deep neural networks where M is a factor that depends on F_j [Bartlett and Mendelson, 2002, Mohri et al., 2012, Bartlett et al., 2017, Kawaguchi

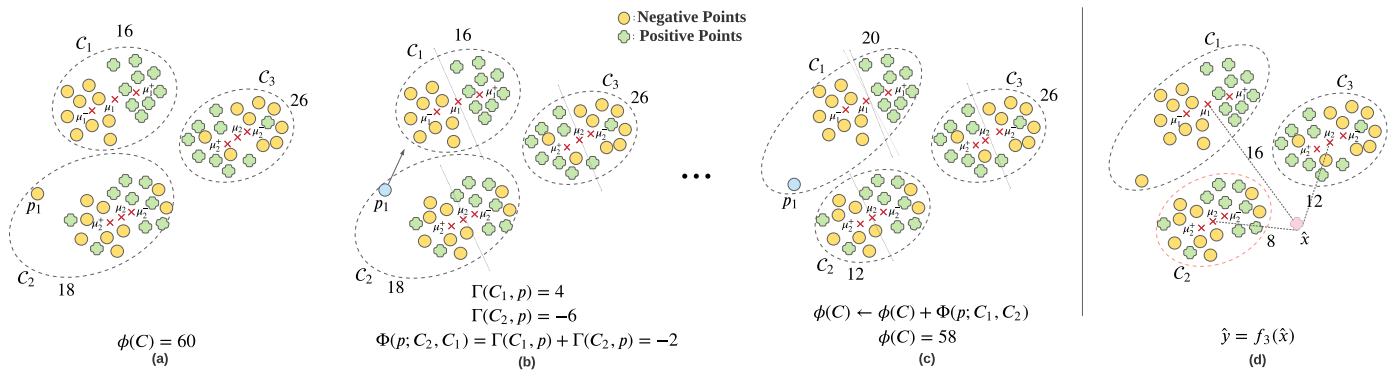


Figure 1: CAC problem setting: (a) The total CAC cost of this clustering is 60; (b) Point p_1 is selected to be reassigned to cluster C_1 based on the cluster update equations; (c) p_1 is assigned to C_1 and the cost functions of C_1 and C_2 are updated; (d) At testing time, $\hat{x}$ is assigned to cluster C_2 as it lies nearest to $\hat{x}$.

et al., 2017]. Since large m reduces both the second and third terms, unsurprisingly, larger sample sizes may lead to improved performance. The theorem shows that simple models within each cluster may lead to good generalization due to smaller values in the first two terms. It thus motivates the formation of clusters, each of which can be linearly separable internally with respect to the classes.

5 CAC: OUR ALGORITHM

In this section, we develop our framework Classification Aware Clustering (CAC). It can be viewed as a clustering algorithm that by design finds clusters that are intended to be used as training datasets by classifiers. The key idea of our approach is to induce class separability within each cluster. If the points of different classes are well separated within a cluster, then it naturally aids any classifier in determining the class boundaries.

5.1 CLASS SEPARABILITY

We use a simple heuristic to measure class separability within each cluster. For each cluster C_j ($j \in [k]$), we define:

- cluster centroid: $\mu(C_j) := |C_j|^{-1} \sum_{x \in C_j}$,
- positive centroid: $\mu^+(C_j) := \frac{1}{\sum_{x_i \in C_j} y_i} \sum_{x_i \in C_j} y_i x_i$, and
- negative centroid: $\mu^-(C_j) := \frac{1}{\sum_{x_i \in C_j} (1-y_i)} \sum_{x_i \in C_j} (1 - y_i) x_i$.

Class separability is defined by considering the distance between positive and negative centroids within each cluster.

Figure 1 shows two clusters C_1 and C_2. C_1 has greater class separation while in C_2, the data points are more intermingled and thus their centroids are closer. Thus, $\|\mu_1^+ - \mu_1^-\| > \|\mu_2^+ - \mu_2^-\|$. From a distributional perspective, if we assume that the data within a cluster follows a Gaussian distribution with a fixed mean and variance, then the number of points that cross a separating hyperplane decreases exponentially as the distance between the classes increases. This is because the density of points decreases exponentially as we move radially away from the center.

5.2 CLUSTERS WITH CLASS SEPARABILITY

To induce our notion of class separability within clustering we consider the k-means clustering formulation that is widely used for its simplicity, effectiveness, and speed. Clusters using k-means are obtained by minimizing the size-weighted sample variance of each cluster:

$$\sum_{p=1}^{k} \inf_{\mu(C_p) \in X} \sum_{i \in C_p} \|x_i - \mu(C_p)\|^2.$$

Thus, the cost of each cluster, given by $\sum_{i \in C_j} \|x_i - \mu(C_j)\|^2$, is minimized.

To induce class separability, a natural formulation is to add a cost with respect to class separability for each cluster. With our measure for class separability, this becomes:

$$\phi(C_j) = \sum_{i \in C_j} \|x_i - \mu(C_j)\|^2 - \alpha \cdot \|\mu^+(C_j) - \mu^-(C_j)\|^2,$$

where $\alpha > 0$ is a class separation hyperparameter. Defining the cost function in this manner encourages the decrease of cluster variance term and increase of the class separability term, weighted by α. Consequently, the overall cost function is defined to be $\phi(\{C_j\}_1^k) := \sum_{j=1}^{k} \phi(C_j)$ as the *unsupervised* loss of CAC that we aim to minimize.

5.3 FINDING THE CLUSTERS

The most common heuristic to find k-means clusters is Lloyd's Algorithm Lloyd [1982]. It begins with an initial clustering and iteratively computes cluster centroids and assigns points to clusters corresponding to their closest centroids. However, using this heuristic with our proposed cost may not guarantee convergence. See our discussion after Theorem 2. Instead, we adopt an approach based on Hartigan's method [Wong and Hartigan, 1979]. For k-means clustering, Hartigan's method proceeds point by point in a greedy manner. Each point is reassigned to a cluster such that the overall cost is reduced.

In our setting, denote the cost of merging a point x to some cluster C_q as $\Gamma^+(C_q,x) = \phi(C_q \cup \{x\}) - \phi(C_q)$. Symmetrically, denote the cost of removing a point x from some cluster C_p as $\Gamma^-(C_p,x) = \phi(C_p \setminus \{x\}) - \phi(C_p)$. Consequently, define the improvement in cost by moving a point $x \in C_p$ from cluster C_p to cluster C_q to be $\Phi(x;C_p,C_q) := \Gamma^+(C_q,x) + \Gamma^-(C_p,x)$.

Our approach is summarized in Algorithm 1. It begins with an initial clustering (e.g., random partitions or k-means) and then proceeds iteratively in rounds. In each round, all points are checked to determine if a point has to be re-assigned to another cluster. For a point in the current iteration, x_i, if moving $x_i \in C_p$ would result in C_p/x_i having points of only one class, then it is not moved. If not, the cluster C_q that results in the maximum decrease in cost $\Phi(x_i;C_p,C_q)$ is found and x_i is moved from its current cluster C_p to another C_q.

In each round, all three centroids for each cluster are updated and classifiers f_j are trained to minimize their corresponding *supervised* training loss $\ell(f,C) = \sum_{j=1}^{k} \ell(f_j,C_j)$. The best collection of classifiers, chosen based on a user-defined performance metric (e.g., training loss or validation set performance), is the final output, along with the corresponding cluster centroids.

We now show that the CAC Algorithm is guaranteed to converge and prove that its per-round time complexity, excluding the cost of classifier training, is linear in the number of data points, data dimensions, and the number of clusters.

Theorem 2. *Algorithm 1 converges to a local minimum.*

Proof. To prove convergence, it suffices to verify that the cost function, that re-assigns point x from cluster C_p to cluster C_q, monotonically decreases at each iteration. This is equivalent to showing

$$\phi(C_p \setminus \{x_i\}) + \phi(C_q \cup \{x_i\}) < \phi(C_p) + \phi(C_q)$$

By definition, this is equivalent to verifying that

$$\Phi(x_i,C_p,C_q) = \Gamma^+(C_q,x) + \Gamma^-(C_p,x) < 0$$

Algorithm 1: CAC

Input: Training Data: $X \in \mathbb{R}^{n\times d}$, binary labels $y^{n\times 1} \in \{0,1\}^n$, $\{C_j\}_1^k$, $\{f_j\}_1^k$ and α
Output: k Trained Classifiers, k cluster centers
▷ **Initialization**
Compute
$\mu(C_j), \mu^+(C_j), \mu^-(C_j)\ \forall\, C_j\ s.t.\ j \in \{1,\cdots,k\}$.
▷ **Algorithm**
while *not converged* **do**
  for $i \in \{1\ldots n\}$ **do**
    if *Removing* $x_i \in C_p$ *from its cluster does not lead to a* 1*-class cluster* **then**
      ▷ **Assign new cluster to** x_i
      Let $q := \arg\min_j \Phi(x_i;C_p,C_j)$ (breaking ties arbitrarily).
      if $\Phi(x_i,C_p,C_q) < 0$ **then**
        Assign x_i to C_q from C_p and update $\mu(C_p)$, $\mu(C_q)$, $\mu(C_p)^{\pm}$ and $\mu(C_q)^{\pm}$.
      Otherwise, let x_i remain in C_p.
  ▷ **Train Classifiers**
  For every cluster C_j, train a classifier f_j taking $x \in C_j$ as the training data.
  ▷ **Retain Best Classifiers:**
  if $\ell(f,C) < \ell(f_{best},C_{best})$ **then**
    $f_{best} = f$
    $\mu_{best} = (\mu,\mu^+,\mu^-)$
    $C_{best} = \{C_j\}_1^k$
Output Classifiers f_{best} and cluster centroids μ_{best}

This is exactly the update condition for the algorithm, and hence, we prove the convergence of CAC. □

Remark 1 (Hartigan vs. Lloyd). For k-means clustering, both Lloyd's method and Hartigan's method converge since the cost function always decreases monotonically with each update. Note that Lloyd's method does not consider the change in centroids while updating the assignment of a point, and instead, directly assign points to their current closest centroids. In our setting, Lloyd's method can similarly fix all centroids, compute a new assignment, and update the centroids accordingly. For instance, replacing the assignment rule of x_i in Algorithm 1 by $q := \arg\min_j \|x_i - \mu(C_j)\|^2 - \alpha \cdot \|\mu^+(C_j) - \mu^-(C_j)\|^2$. However, our cost function contains an additional squared distance term between positive and negative centroids, i.e., $-\alpha \cdot \|\mu^+(C_j) - \mu^-(C_j)\|^2$. There is no guarantee that this term monotonically decreases by Lloyd's method during the update of $\mu^+(C_j)$ and $\mu^-(C_j)$ after an assignment. Hence, it is non-trivial to ensure and prove the convergence of Lloyd's method with our cost function. However, with the use of Hartigan's method, convergence is ensured by design.

Theorem 3. *The time complexity for each round of CAC is $O(ndk+T)$ where T is the time complexity for training a*

set of classifiers f_j over given clusters C_j.

Proof. We present the proof in Appendix A.2. □

5.4 CAC PREDICTION

CAC Algorithm outputs a set of cluster representatives denoted by their centroids and the trained classifiers associated with those clusters. To make predictions for a test point $\hat{x}$, the first step is to find the cluster to which the test point is most likely to belong. This can be done by assigning the test point to cluster represented by the 'closest' centroid C_j in the data space i.e. $j = \arg\min_l \|\hat{x} - \mu_l\|^2$. The corresponding classifier f_j is then used to predict $\hat{y} = f_j(\hat{x})$.

6 EXPERIMENTS

We perform two sets of experiments. The first set of experiments studies, on synthetic data, the sensitivity of CAC to various characteristics of the data. In the second set of experiments, we compare the performance of CAC with that of other methods on real benchmark datasets. All experiments are conducted on a 2.6GHz Intel *i*7 CPU with 16 GB RAM running MAC OS 11.1

Evaluation Metric. In both sets of experiments, F1 score and the Area Under the receiver operating characteristics Curve (AUC) are used as evaluation metrics.

6.1 EXPERIMENTS ON SYNTHETIC DATA

We analyze how the performance of CAC depends on the following parameters associated with the dataset and algorithm.

1. Inner Class Separation (*ICS*): class separation between points in an individual cluster.
2. Outer Cluster Separation (*OCS*): intercluster distance.
3. Number of natural clusters in the dataset (K).
4. Number of classifiers used in CAC (k), i.e., the number of clusters used as input.

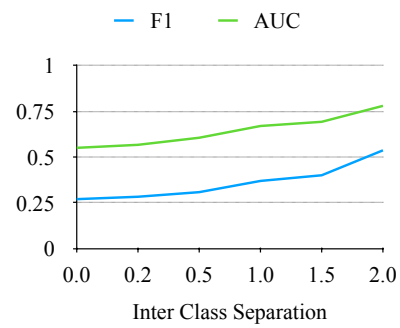
To study the performance of CAC as a function of these 4 parameters, we modify the `make_classification` function of *sklearn* library [Pedregosa et al., 2011], to generate custom datasets with varying ICS, OCS, and K. It distributes points normally about the vertices of a hypercube and assigns an equal number of clusters to each class. We choose Logistic Regression as the base classifier for CAC. We also introduce class imbalance with the ratio of 1 : 10. The range of these parameters in our simulations are as follows: $ICS \in \{0, 0.2, 0.5, 1, 1.5, 2\}$, $OCS \in \{1, 1.5, 2\}$, $K \in \{2, 3, 5, 10, 15, 20, 30\}$ and $k \in \{2, 3, 4, 5\}$. We run CAC+LR for all combination of parameters.

Results

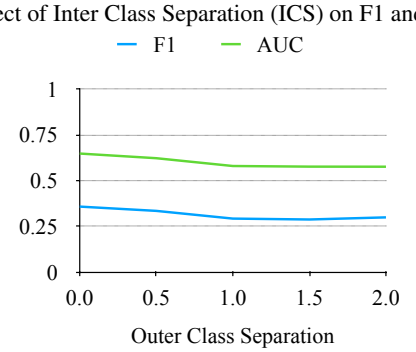
Figures 2 and 3 present the results of the experiments. In each subfigure, the results are averaged over all the parameters not shown in the axes.

In Figure 2a, note that the performance of CAC improves as Inner Class Separation within the clusters increases. This is in line with our hypothesis that if the classes are well separated within the clusters, then it will improve the performance of the classifiers.

The performance of CAC does not depend much on the Outer Cluster Separation (Figure 2b). This is also expected as a classifier will be relatively unaffected by the presence of another cluster, irrespective of its distance as the data in that cluster is not a part of its training set.



(a) Effect of Inter Class Separation (ICS) on F1 and AUC.



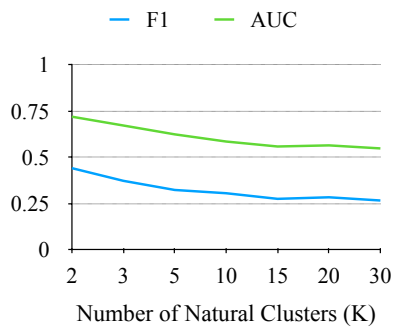
(b) Effect of Outer Cluster Separation (OCS) on F1 and AUC.

Figure 2: Evaluating the effect of different parameters on the performance of CAC

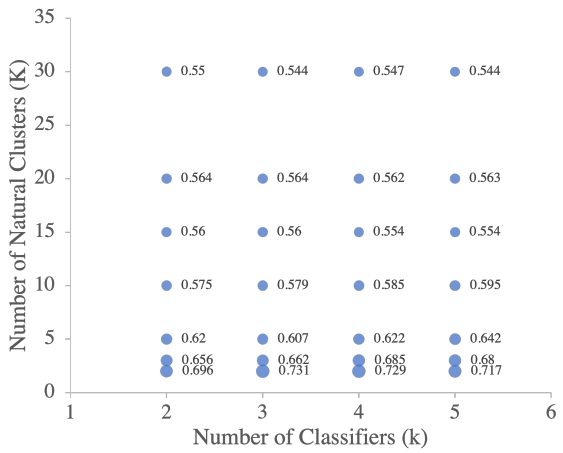
As the number of natural clusters in the dataset increases, the performance of CAC decreases (Figure 3a). This phenomenon is expected from linear classifiers as they cannot handle nonlinearities in data. Hence, dividing the data into multiple groups will decrease the piece-wise nonlinearity and the overall algorithm will work better [Gu and Han, 2013].

In Figure 3b, we observe that the results are better for the cases when $k \geq K$ (i.e., closer to the points near the x-axis). This shows that the performance is better when the number of clusters given as input is not less than the natural clusters in the data, compared to the case when the input number of

clusters is less than the natural number of clusters. In the latter case, each (linear) classifier has to learn a non-linear boundary due to the intrinsic cluster structure that leads to a decrease in the performance.



(a) Effect of number of natural clusters (K) on F1 and AUC.



(b) AUC with varying #classifiers (k) and Natural #Clusters (K)

Figure 3: Evaluating the effect of different parameters on the performance of CAC

In Figure 4, note that the performance of CAC heavily depends on the value of α. If α is too small, then the training will be very slow, and the clusters found will not have enough class separability. If α is too large, CAC will show overfitting behavior. Although the clusters formed will have large class separability and the training error of base classifiers will approach 0, because the clusters are too spread out, at test time, the performance will be poor.

6.2 EXPERIMENTS ON REAL DATASETS

We compare the performance of CAC with state-of-the-art methods for combined clustering and classification on several benchmark datasets. For every dataset, we present results on 5-fold stratified cross-validation.

Datasets We use 5 benchmark datasets that were used to evaluate the EC3 algorithm in [Chakraborty, 2017]. Table 1 presents a summary. We follow the data preprocessing steps

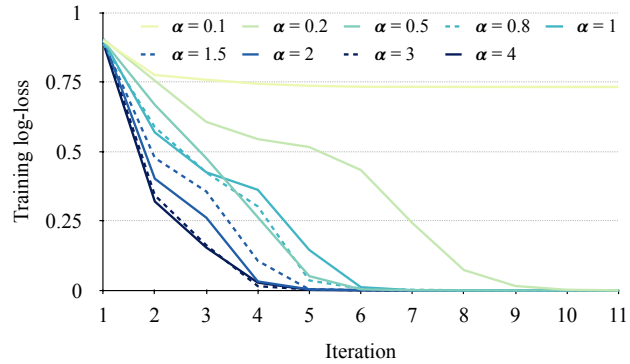


Figure 4: Training Log-Loss with increasing iteration for different values of α

given in [Chakraborty, 2017].

We use two large clinical benchmark datasets, containing clinical data of ICU patients. The Sepsis dataset [Reyna et al., 2019] consists of clinical time-series records of $40,328$ patients. There are 40 time-dependent variables for each patient measured on an hourly basis. A sepsis indicator flag, which is maintained in an hourly manner, is set to 1, six hours before sepsis is detected. We create a static version of the dataset by considering only the record 6 hours before the onset of sepsis for every patient who gets sepsis. For patients who do not get sepsis, we consider their 6th record from the end. Thus our binary classification task is to detect sepsis from clinical signals 6 hours prior to sepsis onset. We use the signature method to extract static features from temporal patterns in the data as used by the winners of the challenge described in [Morrill et al., 2019]. The entire dataset is normalized at the end.

The second clinical dataset, CIC, is from the 2012 Physionet challenge [Silva et al., 2012] to predict in-hospital mortality of ICU patients. The data consists of time-series records from 4,000 ICU stays. We follow the data processing scheme of Johnson et al. [2012] to obtain static features for each patient.

Table 1: Summary of binary classification datasets.

Dataset	#Instances	#Classes	#Features
Titanic [Jr. and Cason, 1994]	2200	2	6
Magic [Lichman., 2013]	19020	2	10
Creditcard [Yeh and Lien, 2009]	30000	2	24
Adult [Lichman., 2013]	45000	2	14
Diabetes [Lichman., 2013]	100000	2	9

Baselines Two state-of-the-art baseline methods for combined clustering and classification are EC3 [Chakraborty, 2017] and DMNN [Li et al., 2020]. EC3 has been shown to outperform 12 other classifiers on these datasets, including popular classifiers like Random Forest and AdaBoost, as well as BGCM [Gao et al., 2011] and UPE [Ao et al., 2014],

Table 2: F1 of each method on different datasets. Averaged over 5 runs. † refers to the best model in the second category while ‡ refers to the best model in DMNN category. The overall best classifier is denoted by *.

Dataset	EC3	LR	*KM*+ LR	**CAC**+ LR	DMNN	**CAC**+ DMNN
Titanic	0.897 ± 0	0.903 ± 0	0.871 ± 0.033	$\mathbf{0.940 \pm 0.011}^{\dagger *}$	0.826 ± 0.022	$\mathbf{0.838 \pm 0.027}^{\ddagger}$
Magic	0.850 ± 0.011	0.848 ± 0.005	$\mathbf{0.858 \pm 0.005}^{\dagger}$	0.858 ± 0.005	$\mathbf{0.905 \pm 0.004}^{\ddagger *}$	0.905 ± 0.004
Credit	$\mathbf{0.494 \pm 0.020}^{*}$	0.360 ± 0.021	$\mathbf{0.486 \pm 0.007}^{\dagger}$	0.486 ± 0.006	0.473 ± 0.018	$\mathbf{0.480 \pm 0.017}^{\ddagger}$
Adult	0.585 ± 0.011	0.553 ± 0.006	0.617 ± 0.008	$\mathbf{0.644 \pm 0.008}^{\dagger}$	$\mathbf{0.465 \pm 0.083}^{\ddagger *}$	0.313 ± 0.211
Diabetes	*NA*	0.118 ± 0.008	0.503 ± 0.006	$\mathbf{0.505 \pm 0.003}^{\dagger *}$	0.235 ± 0.052	$\mathbf{0.251 \pm 0.059}^{\ddagger}$
Sepsis	0.303 ± 0.028	0.369 ± 0.009	0.390 ± 0.009	$\mathbf{0.411 \pm 0.006}^{\dagger}$	0.580 ± 0.015	$\mathbf{0.617 \pm 0.036}^{\dagger *}$
CIC	0.489 ± 0.005	0.443 ± 0.0	0.493 ± 0.012	$\mathbf{0.497 \pm 0.015}^{\dagger}$	0.502 ± 0.009	$\mathbf{0.506 \pm 0.006}^{\ddagger *}$

Table 3: AUC of each method on different datasets. Averaged over 5 runs. † refers to the best model in the second category while ‡ refers to the best model in DMNN category. The overall best classifier is denoted by *.

Dataset	EC3	LR	*KM*+ LR	**CAC**+ LR	DMNN	**CAC**+ DMNN
Titanic	0.986 ± 0	0.980 ± 0	0.980 ± 0.002	$\mathbf{0.998 \pm 0.002}^{\dagger *}$	0.964 ± 0.004	$\mathbf{0.966 \pm 0.005}^{\ddagger}$
Magic	0.842 ± 0.008	0.839 ± 0.006	$\mathbf{0.890 \pm 0.005}^{\dagger *}$	0.889 ± 0.005	0.928 ± 0.004	$\mathbf{0.929 \pm 0.005}^{\ddagger}$
Credit	0.725 ± 0.007	0.723 ± 0.007	$\mathbf{0.741 \pm 0.008}^{\dagger *}$	0.741 ± 0.008	0.770 ± 0.010	$\mathbf{0.771 \pm 0.011}^{\ddagger *}$
Adult	0.802 ± 0.008	0.854 ± 0.002	0.855 ± 0.005	$\mathbf{0.869 \pm 0.005}^{\dagger *}$	$\mathbf{0.817 \pm 0.021}^{\ddagger *}$	0.744 ± 0.117
Diabetes	*NA*	$\mathbf{0.568 \pm 0.002}^{\dagger *}$	0.567 ± 0.003	0.563 ± 0.011	$\mathbf{0.588 \pm 0.005}^{\ddagger *}$	0.588 ± 0.005
Sepsis	0.86 ± 0.006	0.843 ± 0.009	$\mathbf{0.857 \pm 0.012}^{\dagger *}$	0.813 ± 0.020	0.872 ± 0.024	$\mathbf{0.885 \pm 0.014}^{\ddagger *}$
CIC	0.820 ± 0.003	$\mathbf{0.856 \pm 0.0}^{\dagger *}$	0.844 ± 0.005	0.831 ± 0.032	0.840 ± 0.006	$\mathbf{0.843 \pm 0.005}^{\ddagger}$

two consensus maximization approaches that ensemble both classification and clustering methods. DMNN originally employs the k-means algorithm for clustering and a neural network based classifier. In addition, we use Logistic Regression (LR) as a baseline directly and with k-means (KM+LR) in a 'cluster-then-predict' approach. CAC is evaluated with two base classifiers LR and DMNN. In the latter case, k-means is replaced with CAC, and DMNN's underlying neural architecture is used.

Hyperparameters For all KM+X and CAC+X experiments in Tables 2 and 3, we choose $k = 2$. For CAC, we set α according to the dataset the details of which can be found in Appendix A.3. For DMNN, the autoencoder has 3 layer of sizes $64 - 32 - 64$ and a two-layered local predictor network of size $30 - 1$. The autoencoder has L1-L2 regularization (L1: 1e-5, L2: 1e-4) and no activation. The local prediction networks use softmax and relu activation functions and are not regularized.

Results Table 2 presents the F1 scores obtained by all algorithms. On the Diabetes dataset, EC3 was unable to finish even after 2 hours. We see that CAC+LR outperforms, or has a very similar performance to, LR and KM+LR in 5 out of 7 datasets. A similar trend can be seen in CAC+DMNN where it outperforms DMNN in 5 out 7 datasets. In 6 out of 7 datasets either CAC+LR or CAC+DMNN outperforms EC3. Table 3 presents the AUC scores on these datasets where we observe similar trends.

7 CONCLUSION

Clustering has been effectively used with classification models in many previous models and for various applications. In this paper, we theoretically analyze potential reasons for the good performance of classifiers trained on underlying clusters in data. Our analysis yields new insights into the benefits of using simple classifiers trained on clusters compared to simple or complex classifiers trained without clustering. A deeper analysis along this direction would be an interesting direction for future work.

Our analysis motivates the development of a new clustering framework, CAC, that enforces class separability within each cluster during cluster discovery. We design an efficient iterative algorithm to find such clusters. Unlike most previous such approaches, CAC is simple, efficient, and can be used with any classifier. In our experiments, we find that CAC outperforms state-of-the-art methods of combined clustering and classification. Extensions of our approach to multi-class classification and more efficient methods to find the best classifiers on the clusters may be investigated in the future.

References

A. M. Alaa, J. Yoon, S. Hu, and M. Van der Schaar. Personalized risk scoring for critical care patients using mixtures of gaussian process experts. *arXiv preprint arXiv:1605.00959*, 2016.

I. Alsmadi and I. Alhami. Clustering and classification of email contents. *Journal of King Saud University-Computer and Information Sciences*, 2015.

K. S. Ambrosen, K. J. Albers, T. B. Dyrby, M. N. Schmidt, and M. Mørup. Nonparametric bayesian clustering of structural whole brain connectivity in full image resolution. In *PRNI*, 2014.

X. Ao, P. Luo, X. Ma, F. Zhuang, Q. He, Z. Shi, and Z. Shen. Combining supervised and unsupervised models via unconstrained probabilistic embedding. *Information Sciences*, 2014.

P. L. Bartlett and S. Mendelson. Rademacher and gaussian complexities: Risk bounds and structural results. *JMLR*, 2002.

P. L. Bartlett, D. J. Foster, and M. J. Telgarsky. Spectrally-normalized margin bounds for neural networks. In *NeurIPS*, 2017.

W. H. Beluch, T. Genewein, A. Nürnberger, and J. M. Köhler. The power of ensembles for active learning in image classification. In *CVPR*, 2018.

W. Cai, S. Chen, and D. Zhang. A simultaneous learning framework for clustering and classification. *Pattern Recognition*, 2009.

T. Chakraborty. Ec3: Combining clustering and classification for ensemble learning. In *ICDM*, 2017.

S. Dasgupta. *The hardness of k-means clustering*. University of California, 2008.

C. Durkan, I. Murray, and G. Papamakarios. On contrastive learning for likelihood-free inference. In *ICML*, 2020.

H. Elouedi, W. Meliani, Z. Elouedi, and N. B. Amor. A hybrid approach based on decision trees and clustering for breast cancer classification. In *SoCPaR*, 2014.

Z. Fu, A. Robles-Kelly, and J. Zhou. Mixing linear svms for nonlinear classification. *IEEE Transactions on Neural Networks*, 2010.

J. Gao, F. Liang, W. Fan, Y. Sun, and J. Han. A graph-based consensus maximization approach for combining multiple supervised and unsupervised models. *TKDE*, 2011.

Q. Gu and J. Han. Clustered support vector machines. In *Artificial Intelligence and Statistics*, 2013.

B. Hao, W. W. Sun, Y. Liu, and G. Cheng. Simultaneous clustering and estimation of heterogeneous graphical models. *JMLR*, 2018.

D. Heider, R. Senge, W. Cheng, and E. Hüllermeier. Multilabel classification for exploiting cross-resistance information in hiv-1 drug resistance prediction. *Bioinformatics*, 2013.

S. Hess, W. Duivesteijn, and D. Mocanu. Softmax-based classification is k-means clustering: Formal proof, consequences for adversarial attacks, and improvement through centroid based tailoring. *arXiv:2001.01987*, 2020.

Z. Hussain, F. Gimenez, D. Yi, and D. Rubin. Differential data augmentation techniques for medical imaging classification tasks. In *AMIA Annual Symposium*, 2017.

Z. M. Ibrahim, H. Wu, A. Hamoud, L. Stappen, R. J. Dobson, and A. Agarossi. On classifying sepsis heterogeneity in the icu: insight using machine learning. *Journal of the American Medical Informatics Association*, 2020.

X. Jiang, L. Ding, M. Havaei, A. Jesson, and S. Matwin. Task adaptive metric space for medium-shot medical image classification. In *MICCAI*, 2019.

A. E. Johnson, N. Dunkley, L. Mayaud, A. Tsanas, A. A. Kramer, and G. D. Clifford. Patient specific predictions in the intensive care unit using a bayesian ensemble. In *CinC*, 2012.

F. E. H. Jr. and T. Cason. Titanic, 1994. URL `https://www.openml.org/d/40945`.

K. Kawaguchi, L. P. Kaelbling, and Y. Bengio. Generalization in deep learning. *arXiv:1710.05468*, 2017.

T.-K. Kim and R. Cipolla. Mcboost: Multiple classifier boosting for perceptual co-clustering of images and visual features. *Advances in Neural Information Processing Systems*, 21:841–848, 2008.

X. Li, D. Zhu, and P. Levy. Predicting clinical outcomes with patient stratification via deep mixture neural networks. *AMIA Summits on Translational Science*, 2020.

M. Lichman. Uci repository, 2013. URL `http://archive.ics.uci.edu/ml`.

S. Lloyd. Least squares quantization in pcm. *IEEE transactions on information theory*, 1982.

A. Mahfouz, A. Abuhussein, D. Venugopal, and S. Shiva. Ensemble classifiers for network intrusion detection using a novel network attack dataset. *Future Internet*, 2020.

M. Mohri, A. Rostamizadeh, and A. Talwalkar. *Foundations of machine learning*. 2012.

J. Morrill, A. Kormilitzin, A. Nevado-Holgado, S. Swaminathan, S. Howison, and T. Lyons. The signature-based model for early detection of sepsis from electronic health records in the intensive care unit. In *CinC*, 2019.

S. Patel, C. Mancinelli, J. Healey, M. Moy, and P. Bonato. Using wearable sensors to monitor physical activities of patients with copd: A comparison of classifier performance. In *BSN*, 2009.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, et al. Scikit-learn: Machine learning in Python. *JMLR*, 2011.

M. Peikari, S. Salama, S. Nofech-Mozes, and A. L. Martel. A cluster-then-label semi-supervised learning approach for pathology image classification. *Scientific reports*, 2018.

G. Qi, Q. Tian, and T. Huang. Locality-sensitive support vector machine by exploring local correlation and global regularization. In *CVPR*, 2011.

Q. Qian, S. Chen, and W. Cai. Simultaneous clustering and classification over cluster structure representation. *Pattern Recognition*, 2012.

M. A. Reyna, C. Josef, S. Seyedi, R. Jeter, S. P. Shashikumar, M. B. Westover, A. Sharma, S. Nemati, and G. D. Clifford. Early prediction of sepsis from clinical data: the physionet/computing in cardiology challenge. In *CinC*, 2019.

H. Sasaki, T. Takenouchi, R. Monti, and A. Hyvarinen. Robust contrastive learning and nonlinear ica in the presence of outliers. In *UAI*, 2020.

I. Silva, G. Moody, D. J. Scott, L. A. Celi, and R. G. Mark. Predicting in-hospital mortality of icu patients: The physionet/computing in cardiology challenge 2012. *Computing in cardiology*, 2012.

R. F. Silva, E. Castro, C. N. Gupta, M. Cetin, M. Arbabshirani, V. K. Potluru, S. M. Plis, and V. D. Calhoun. The tenth annual mlsp competition: schizophrenia classification challenge. In *MLSP*, 2014.

R. Snyderman. Personalized health care: from theory to practice. *Biotechnology journal*, 2012.

K. Song, F. Nie, J. Han, and X. Li. Parameter free large margin nearest neighbor for distance metric learning. In *AAAI*, 2017.

H. Suresh, J. J. Gong, and J. V. Guttag. Learning tasks for multitask learning: Heterogenous patient populations in the icu. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018.

A. K. Tanwani and M. Farooq. The role of biomedical dataset in classification. In *AIME*, 2009.

I. Titov, A. Klementiev, K. Small, and D. Roth. Unsupervised aggregation for classification problems with large numbers of categories. In *AISTATS*, 2010.

P. H. Torr. Locally linear support vector machines. In *ICML*, 2011.

M. Vincent and N. R. Hansen. Sparse group lasso and high dimensional multinomial classification. *Computational Statistics & Data Analysis*, 2014.

A. Visentin, A. Nardotto, and B. O'Sullivan. Predicting judicial decisions: A statistically rigorous approach and a new ensemble classifier. In *ICTAI*, 2019.

K. Q. Weinberger and L. K. Saul. Distance metric learning for large margin nearest neighbor classification. *JMLR*, 2009.

M. A. Wong and J. Hartigan. Algorithm as 136: A k-means clustering algorithm. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 1979.

I.-C. Yeh and C.-h. Lien. The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 2009.

A APPENDIX

A.1 PROOF OF THEOREM 1

While Theorem 1 is a direct application of a previous result [Bartlett and Mendelson, 2002, Mohri et al., 2012] to our problem, we provide a proof of Theorem 1 based on a previous work [Mohri et al., 2012] for completeness:

Proof of Theorem 1. Let $S = ((x_i, y_i))_{i=1}^m$ and $S' = ((x'_i, y'_i))_{i=1}^m$. Define

$$\varphi(S) = \sup_{f \in \mathscr{F}_j} \mathbb{E}_{x,y}[\ell(f(x), y)] - \frac{1}{m}\sum_{i=1}^m \ell(f(x_i), y_i). \quad (2)$$

To apply McDiarmid's inequality to $\varphi(S)$, we compute an upper bound on $|\varphi(S) - \varphi(S')|$ where S and S' be two test datasets differing by exactly one point of an arbitrary index i_0; i.e., $S_i = S'_i$ for all $i \neq i_0$ and $S_{i_0} \neq S'_{i_0}$. Then,

$$\varphi(S') - \varphi(S) \leq \sup_{f \in \mathscr{F}_j} \frac{\ell(f(x_{i_0}), y_{i_0}) - \ell(f(x'_{i_0}), y'_{i_0})}{m} \leq \frac{\lambda}{m}. \quad (3)$$

Thus, by McDiarmid's inequality, for any $\delta > 0$, with probability at least $1-\delta$,

$$\varphi(S) \leq \mathbb{E}_S[\varphi(S)] + \lambda\sqrt{\frac{\ln(1/\delta)}{2m}}. \tag{4}$$

Moreover,

$$\begin{aligned}
&\mathbb{E}_S[\varphi(S)] && (5)\\
&= \mathbb{E}_S\left[\sup_{f\in\mathscr{F}_j}\mathbb{E}_{S'}\left[\frac{1}{m}\sum_{i=1}^m \ell(f(x_i'),y_i')\right] - \frac{1}{m}\sum_{i=1}^m \ell(f(x_i),y_i)\right] && (6)\\
&\leq \mathbb{E}_{S,S'}\left[\sup_{f\in\mathscr{F}_j}\frac{1}{m}\sum_{i=1}^m (\ell(f(x_i'),y_i') - \ell(f(x_i),y_i)\right] && (7)\\
&\leq \mathbb{E}_{\xi,S,S'}\left[\sup_{f\in\mathscr{F}_j}\frac{1}{m}\sum_{i=1}^m \xi_i(\ell(f(x_i'),y_i') - \ell(f(x_i),y_i))\right] && (8)\\
&\leq 2\mathbb{E}_{\xi,S}\left[\sup_{f\in\mathscr{F}_j}\frac{1}{m}\sum_{i=1}^m \xi_i\ell(f(x_i),y_i))\right] = 2\mathscr{R}_m(\ell\circ\mathscr{F}_j) && (9)
\end{aligned}$$

where the first line follows the definitions of each term, the second line uses the Jensen's inequality and the convexity of the supremum, and the third line follows that for each $\xi_i \in \{-1,+1\}$, the distribution of each term $\xi_i(\ell(f(x_i'),y_i') - \ell(f(x_i),y_i))$ is the distribution of $(\ell(f(x_i'),y_i') - \ell(f(x_i),y_i))$ since S and S' are drawn iid with the same distribution. The forth line uses the subadditivity of supremum. □

A.2 PROOF OF THEOREM 3

Proof. For the time complexity of each round, it suffices to prove the running time for each point x_i is $O(dk)$. For each $j \in [k]$, we denote the centroid, positive centroid and negative centroid of $C_j \cup \{x_i\}$ as $\tilde{\mu}(C_j)$, $\tilde{\mu}^+(C_j)$ and $\tilde{\mu}^-(C_j)$ respectively. Given a cluster C_j and a point $z \in \mathbb{R}^d$, we denote the clustering cost of C w.r.t. z to be

$$g(C_j,z) := \sum_{x\in C_j}\|x-z\|^2.$$

A standard result from the k-means literature [Dasgupta, 2008] is the following bias-variance decomposition of the k-means cost function:

$$g(C_j,z) = \phi(C_j,\mu(C_j)) + |C_j|\cdot\|\mu(C_j)-z\|^2.$$

Note that the above result holds for CAC cluster cost function also as can be shown easily by a little algebra. Based on this property, we have

$$\begin{aligned}
&\Gamma^+(C_j,x_i)\\
=&\|\tilde{\mu}-x_i\|^2 + |C_j|\cdot\|\tilde{\mu}(C_j)-\mu(C_j)\|^2\\
&+\alpha|C_j|\cdot\|\mu^+(C_j)-\mu^-(C_j)\|^2\\
&-\alpha(|C_j|+1)\cdot\|\tilde{\mu}^+(C_j)-\tilde{\mu}^-(C_j)\|^2\\
&\Gamma^-(C_j,x_i)\\
=&-\|\mu(C_j)-x_i\|^2 - (|C_j|-1)\|\tilde{\mu}(C_j)-\mu(C_j)\|^2\\
&+\alpha|C_j|\cdot\|\mu^+(C_j)-\mu^-(C_j)\|^2\\
&-\alpha(|C_j|-1)\cdot\|\tilde{\mu}^+(C_j)-\tilde{\mu}^-(C_j)\|^2
\end{aligned}$$

Then it suffices to prove that both $\Gamma^+(C_j,x_i)$ and $\Gamma^-(C_j,x_i)$ can be computed in $O(d)$ time, which implies $O(dk)$ time for computing $q := \arg\min_j \Phi(x_i;C_p,C_j)$. Moreover, by the above formulations, it suffices to prove that $\tilde{\mu}(C_j)$, $\tilde{\mu}^+(C_j)$ and $\tilde{\mu}^-(C_j)$ can be computed in $O(d)$ time. Note that $\tilde{\mu}(C_p) = \frac{|C_p|\cdot\mu(C_p)-x_i}{|C_p|-1}$ and $\tilde{\mu}(C_j) = \frac{|C_j|\cdot\mu(C_j)+x_i}{|C_j|+1}$ for $j \neq p$. Hence, each $\tilde{\mu}(C_j)$ can be computed in $O(d)$ time. We discuss the following two cases for the computation of $\tilde{\mu}^+(C_j)$ and $\tilde{\mu}^-(C_j)$.

- Case $y_i = 1$. We have that $\tilde{\mu}^+(C_p) = \frac{\left(\sum_{x_l\in\mathscr{C}_p} y_l\right)\cdot\mu_p^+ - x_i}{\sum_{x_l\in\mathscr{C}_p} y_l - 1}$ and $\tilde{\mu}^-(C_p) = \mu^-(C_p)$. For $j \neq p$, we have that $\tilde{\mu}^+(C_j) \leftarrow \frac{\left(\sum_{x_l\in\mathscr{C}_j} y_l\right)\cdot\mu_j^+ + x_i}{\sum_{x_l\in\mathscr{C}_j} y_l + 1}$ and $\tilde{\mu}^-(C_j) = \mu^-(C_j)$.
- Case $y_i = 0$. We have that $\tilde{\mu}_p^- \leftarrow \frac{\left(\sum_{x_l\in\mathscr{C}_p}(1-y_l)\right)\cdot\mu_p^- - x_i}{\sum_{x_l\in\mathscr{C}_p}(1-y_l) - 1}$ and $\tilde{\mu}^+(C_p) = \mu^+(C_p)$. For $j \neq p$, we have that $\tilde{\mu}_j^- \leftarrow \frac{\left(\sum_{x_l\in\mathscr{C}_j}(1-y_l)\right)\cdot\mu_j^- + x_i}{\sum_{x_l\in\mathscr{C}_j}(1-y_l) + 1}$ and $\tilde{\mu}^+(C_j) = \mu^+(C_j)$.

By the above update rules, both $\tilde{\mu}^+(C_j)$ and $\tilde{\mu}^-(C_j)$ can be computed in $O(d)$ time, which completes the proof. □

A.3 α VALUES FOR CAC

Table 4: α for different datasets

Dataset	α
Titanic	100
Magic	0.04
Creditcard	0.04
Adult	0.1
Diabetes	2
Sepsis	0.04
CIC	0.04